ASSIGNMENT 5 – CO5

PART A | TASK A4

WHAT SHOULD BE SERIALIZED?

Serialization, Object-Graph Persistence and Version Compatibility

Course: Programming in Java
Topic: Data Persistence Engineering – Files and Databases

A design analysis of transient fields, recursive object-graph serialization, serialVersionUID compatibility, and an alternative CSV/JSON persistence approach.

1. Problem Understanding

The hostel management program serializes a Hostel object. Hostel contains a list of Room objects, and each Room contains a list of Resident objects. Each Resident contains a name, register number, password and a reference to a live database Connection. The design must separate durable application data from secrets and runtime resources, then explain how the saved data behaves when the Resident class changes.

2. A4.1 – Decide What to Persist

Recommended field decision

Field	Persist?	Reason
Hostel.rooms	Yes	Contains the collection of room records required to restore hostel state.
Room.residents	Yes	Contains the resident records belonging to each room.
Resident.name	Yes	Ordinary resident information needed after restart.
Resident.registerNumber	Yes	Stable identifier needed to identify the resident.
Resident.password	No – transient	A credential must not be copied into a general serialized data file. Prefer authentication through a secure credential store; never persist a plaintext password in this object snapshot.
Resident.connection	No – transient	A live JDBC Connection is a runtime resource tied to a driver, session and database state. It is not durable application data and should be recreated when needed.

Illustrative Java field declarations

```
class Resident implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private String registerNumber;

    private transient String password;
    private transient Connection connection;
}
```

The classes Hostel, Room and Resident must all implement Serializable (or otherwise be be handled through a custom serialization strategy) for default serialization to traverse the complete graph. The collections used for the lists must also be serializable.

What happens when Hostel is serialized?

ObjectOutputStream starts at the Hostel reference and follows its non-transient, serializable instance fields. It therefore writes the Hostel object, its Room list, each Room object, its Resident list and each reachable Resident object. This is recursive object-graph serialization, not merely writing the Hostel fields in isolation. Java serialization preserves object references and shared references within the stream; it can also handle cycles in the graph. If a reachable non-transient object is not serializable, serialization fails with NotSerializableException.

The transient password and Connection fields are skipped. After deserialization, reference fields that were transient are set to their default value, null. The application must re-authenticate or obtain credentials securely and create a fresh database connection through its connection manager; it must not expect the old live connection to return.

3. A4.2 – Plan for Change: Adding phoneNumber

Assume phoneNumber is added to Resident after an older Hostel snapshot has already been saved. When the old stream is read by the new class version, Java checks the stream class descriptor against the local class descriptor, including serialVersionUID.

If serialVersionUID is explicitly kept the same

If the new Resident class declares the same serialVersionUID as the class used to write the old file, the old stream can normally be read. The old stream has no phoneNumber value, so the newly added field receives its Java default value: null for String, 0 for numeric primitives, false for boolean. The application should handle the missing value, for example by treating it as not provided and prompting for an update.

If serialVersionUID is different or incompatible

If the local class has a different serialVersionUID from the stream descriptor, deserialization fails with InvalidClassException. If no serialVersionUID is declared, Java computes one from class details; changing the class can change that computed value and break compatibility. Declaring an explicit UID makes compatibility decisions deliberate, but it does not guarantee that every structural change is safe. Developers must still test the evolution of serialized classes and validate old data.

Recommended compatibility practice

Declare an explicit serialVersionUID, keep it unchanged only when the class change is serialization-compatible, and define defaults or migration logic for newly introduced fields. For incompatible changes, use a controlled migration process or a versioned external format. Never change the UID merely to suppress an exception without checking the data layout.

4. Alternative Concept – Store Hostel Data as CSV or JSON

Instead of Java's object serialization, the application can write one resident record per CSV row or JSON object. The file should contain only durable, permitted fields, such as name, register number and phone number. Passwords and live database connections should not be exported. Room and Hostel relationships can be represented using room IDs and resident records, or nested JSON arrays/objects.

Aspect	Java serialization	CSV / JSON
Representation / size	Usually compact binary for many primitive/object values, but includes class and reference metadata; size depends on graph and strings.	Text is readable and often larger due to field names, delimiters, quotes and textual numbers; compression can reduce storage.
Save n residents	O(n) traversal and writing for n resident objects and their reachable data.	O(n) when serializing/writing each resident record.
Load n residents	O(n) traversal and reconstruction of the object graph.	O(n) parsing and reconstruction of n records.
Memory during load	Typically O(n) to reconstruct and hold the resulting object graph.	O(1) extra working memory is possible with streaming parsers; loading the whole document/list requires O(n).
Adding phoneNumber	Requires class-version compatibility handling; absent field gets its default if UID compatibility permits.	CSV needs a header/column or schema update and missing-column defaults; JSON parsers can treat a missing property as optional/default.
Portability / coupling	Java-specific format and class definitions; convenient for Java object graphs but tightly coupled to class evolution.	More language-independent and human-readable; requires explicit parsing, validation and relationship reconstruction.

Evaluation and decision

Both approaches require O(n) time to save and load n resident records because each record must be written or parsed. The principal difference is representation and compatibility, not the Big-O time. Java serialization is convenient when the application needs to restore a Java object graph with shared references, but it is Java-specific and class-version sensitive. CSV is suitable for flat tabular records and simple data exchange. JSON is better when nested Room/Resident relationships must be represented. A streaming CSV/JSON parser can keep extra working memory near O(1) (excluding the application data retained); parsing the entire file into a collection uses O(n) memory. For long-term data exchange or frequent schema changes, versioned JSON/CSV with explicit validation and migration is generally easier to evolve than native Java serialization.

5. Conclusion

Persist the Hostel, Room and Resident structure and the durable resident fields name and register number. Mark password and live Connection transient: the password is sensitive and the connection is a runtime resource that must be recreated. Serialization traverses the reachable serializable object graph. When phoneNumber is added, an old stream read with a compatible explicit serialVersionUID gives the new field its default value; an incompatible UID causes InvalidClassException. CSV/JSON also take O(n) time to save and load n records, but offer a more portable and explicitly evolvable format.

6. Submission Note

Task A4 belongs to Part A – Persistence Design & Pseudocode. It asks for analysis and design reasoning, not a complete runnable Java program. Therefore no Java source file, compiler command or execution screenshot is required specifically for this task. The illustrative field declarations are included to make the transient decision clear.